INDIAN INSTITUTE OF TECHNOLOGY KANPUR

CampusEcho

Software Engineering Project — Deliverable 3

Testing, Security, Concurrency, Reliability, Performance and Deployment

Course:	CS455 — Introduction to Software Engineering	
Project:	CampusEcho (Anonymous Course Feedback)	
Repository:	github.com/Kishore200524/CS455-Project	
Team members:	M. Amara Gayathri	230624
	Reddi Pallavi	230850
	Poorvie Sadagopan	230759
	Bhukya Vaishnavi	230295
	Kishore Senthilkumar	230566
	Vundela Obula Reddy	231178

Contents

1 Executive Summary 2

2 System Under Test and Evidence Provenance 2

3 Unit, Integration and Automated Test Results 2

4 Security and Adversarial Evaluation 3

5 Concurrency Experiments and Atomicity 4

6 Failure Injection, Recovery and Reliability 5

7 Performance and Load Testing 7

8 System and End-to-End Validation 8

9 Defects, Fixes and Regression Traceability 8

10 Cloud Deployment and Operational Validation 10

11 Cost Estimation 12

12 Planning, Requirements Traceability, Jira and AI Engineering Log 13

13 Conclusion and Limitations 18

A Evidence Folder Index 18

1 Executive Summary

CampusEcho is an anonymous course-feedback platform with student review submission, course discovery, announcements, AI-assisted triage, a shared moderator appeal queue, and student appeals. The implemented stack comprises a React/TypeScript frontend, Go HTTP API, MongoDB persistence, and an asynchronous Gemini triage pipeline. Controls include server-side role checks, atomic moderation claims, session management, triage worker leases, and recovery safeguards.

Deliverable 3 produced a successful backend suite (81.0% overall statement coverage), full-source frontend testing (163 tests, 87.53% statement coverage), a passing race-detector run, adversarial tests, two concurrency experiments, failure/recovery tests, and an authenticated course-search baseline of 18,135 successful requests in 30.16 seconds. Five integrated E2E journeys were additionally confirmed by the team.

2 System Under Test and Evidence Provenance

The frontend presents student, moderator, and administrator flows. The Go backend authenticates users, enforces roles, manages reviews and appeal tickets, and communicates with MongoDB. A worker pool performs asynchronous AI triage, with configurable Gemini or development-mock providers and background recovery scanning. Tests span pure unit checks, HTTP handlers, MongoDB-backed integration tests, and controlled provider-failure injection.

Evidence convention. Linked paths in this report are repository-relative beneath the evidence folder. A passing automated test is an observed result for the behavior it asserts; it is not, by itself, a live-production demonstration.

3 Unit, Integration and Automated Test Results

3.1 Backend tests and MongoDB integration

The saved `01-automated-tests/backend-tests.txt` reports success across the test-bearing Go packages, including authentication, HTTP API, database, appeals, triage, courses, announcements and feedback; the command exited with code 0. Command entry-point packages without test files are not represented as tested CLI entry points. MongoDB-backed integration coverage includes appeal-resolution transactions, queue claims, stale-worker protection and provider cooldown.

3.2 Statement coverage

The full Go coverage run exited successfully and reported **81.0% overall statement coverage** (81.01% from detailed local counts). Entry-point packages with zero coverage are included, so the overall figure is not limited to highly tested modules.

Backend package	Statements	Backend package	Statements
`internal/httpapi`	91.0%	`internal/database`	85.7%
`internal/auth`	84.9%	`internal/triage`	87.6%
`internal/appeals`	83.8%	`internal/feedback`	96.3%
`internal/config`	95.0%	`internal/courses`	100.0%
`internal/announcements`	100.0%	`cmd/seed-courses`	39.5%

Table 1: Verified package coverage from the final repository evidence.

Evidence: `01-automated-tests/backend-coverage-run.txt`, `01-automated-tests/backend-coverage.txt`.

3.3 Frontend tests, build and dependency review

The full-source Vitest/V8 run reports **23 passing files, 163 passing tests**, 87.53% statements, 85.23% branches, 83.51% functions and 88.79% lines. The production frontend build passed; `npm audit` reported zero vulnerabilities at the time of execution. These consolidated results supersede earlier component-specific coverage figures and earlier teammate snapshots.

Metric	Final value	Metric	Final value
Frontend tests	163 passed	Test files	23 passed
Statements	87.53%	Branches	85.23%
Functions	83.51%	Lines	88.79%

Table 2: Final consolidated frontend results.

Contributions across modules. PR #11 introduced the frontend Vitest/React Testing Library infrastructure and 50 tests across eight files, plus four HTTP integration tests. These cover course selection, announcements, My Codes, session-cookie recovery, empty search and course-ID validation. PR #18 added moderator component/service tests for queue pages, ticket detail, modal behavior, active appeals and API integration. PR #10 contributes duplicate/brigading and retry-worker tests; PR #13 contributes Gemini and persisted-recovery tests; PR #15–PR #17 add OTP/authentication security regressions. Counts from individual PRs are *subsets* of the final 163 tests, not additive totals.

Evidence: `01-automated-tests/frontend-coverage.txt`, `01-automated-tests/frontend-build.txt`, `01-automated-tests/npm-audit.txt`; tests and changes in PR #10, PR #11, PR #13, PR #15, PR #16, PR #17, PR #18.

3.4 Race detector

The final `go test -race ./... -count=1` run passed with exit code 0 for all test-bearing backend packages. The earlier `backend-race.txt` records an intermediate failed diagnostic run; the final result is `01-automated-tests/backend-race-final.txt`.

4 Security and Adversarial Evaluation

4.1 Threat model

ID	Threat / asset	Impact	Defense and verification
TH-01	Student accesses moderator APIs	Unauthorized access	Server-side moderator role checks; moderator token/body regression tests; manual 403.
TH-02	Moderator changes another moderator's appeal	Unauthorized ticket state change	Lock-holder checks, transactions and integration tests; manual non-holder 403.
TH-03	Concurrent OTP guessing	Five-attempt limit bypass	Atomic attempt reservation in MongoDB (PR #15); 100-request concurrency regression.
TH-04	Login/OTP brute forcing	Credential compromise, email abuse	Account/IP throttling and privileged password minimum (PR #16); auth tests.
TH-05	Account enumeration	Leaking registered identities/roles	Uniform credential-failure response and dummy bcrypt work for unknown accounts (PR #17); auth tests.

ID	Threat / asset	Impact	Defense and verification
TH-06	Course-ID variants/forged titles	Bypassing course-specific spam limits	Canonical catalog IDs/titles and rejection of unlisted courses (PR #11); three backend and frontend regressions.
TH-07	Cookie/bearer confusion and cross-student browser state	Role confusion or reference-code mix-up	Bearer precedence test and cookie-aware frontend; student lookup not cached across cookie-only users (PR #11).
TH-08	Malformed input or AI output	Unexpected mutation/publication	Strict request/comment validation; Gemini decision validation and deferred recovery (PR #13, PR #18).

4.2 Three manual security scenarios

The team confirmed the following outcomes on the running application. These are reported as *manual observations*; raw requests/screenshots were not saved as separate evidence.

ID	Operation	Expected	Observed
SEC-01	Student requests moderator-only endpoint	403	403, passed
SEC-02	Moderator B reads/releases A's locked ticket	403	403, passed
SEC-03	Whitespace-only deletion comment	400	400, passed

Table 4: Team-confirmed manual security checks.

4.3 Adversarial, negative and regression tests

The pushed logs verify moderator-route authorization, malformed body/invalid ID/wrong-method handling, cookie-over-bearer precedence, auth-error non-disclosure, login/OTP throttling and replay checks. The contributed team documents add course-ID variant attempts (`CS455`, whitespace variants and unlisted course IDs), cookie-only request behavior, corrupt browser storage, and the cross-student notebook/announcement isolation checks. PR #18 expands moderator HTTP handler and UI adversarial coverage. A teammate's *break-it* mutation checks deliberately altered authorization and sort-validation behavior to confirm that the corresponding tests fail, then reverted those temporary changes. These are development assertions, not an independent penetration test.

Residual risks. The catalog must be populated for the strict course-validation rule to permit submissions; browser-profile local storage remains accessible to people sharing that profile; process-local rate limiting requires distributed/deployment review. These limitations should be evaluated in the deployment threat model.

Evidence: `02-security/httpapi-security-tests.txt`, `02-security/httpapi-auth-tests.txt`, `02-security/auth-security-tests.txt`; PR #11, PR #15, PR #16, PR #17, PR #18.

5 Concurrency Experiments and Atomicity

5.1 Shared appeal ticket claiming

The repository test `TestQueueClaimConcurrentRequests` creates one claimable ticket per round, then starts **50 concurrent claimant goroutines** using a shared start barrier. The test repeats **20 rounds**, representing **1,000 total claim requests**. Each round asserts exactly one successful claim and 49 `ErrAlreadyClaimed` results, with no unexpected errors. The saved test passed (exit code 0). This establishes the at-most-one-winner property under the tested MongoDB integration configuration; it is not a throughput benchmark.

5.2 Concurrent OTP verification

`TestOTPVerificationReservesAtMostFiveConcurrentAttempts` simultaneously submits **100** invalid OTP verification requests against an in-memory test repository. It asserts that exactly five requests consume the permitted attempts, 95 are rejected due to the attempt limit, and the stored attempt count stays at five. The test passed. This complements, but does not replace, the MongoDB-backed moderation experiment.

Experiment	Safety invariant	Load	Result
Concurrent appeal claims	Exactly one winner per ticket; 49 conflict responses	50 claimants × 20 rounds	PASS
Concurrent OTP guesses	At most five reserved attempts	100 simultaneous attempts	PASS

Table 5: Concurrency correctness experiments (not latency measurements).

Evidence: `03-concurrency/concurrent-ticket-claim.txt` and `03-concurrency/concurrent-otp-verification.txt`.

6 Failure Injection, Recovery and Reliability

This section integrates reliability testing across *AI workers, MongoDB moderation transactions, authentication and student/moderator interfaces*. All automated claims below are supported by the pushed logs or named regression tests in merged pull requests. The scenarios are controlled injections or negative-path tests; none should be mistaken for a deliberately induced live cloud outage.

6.1 FR-01: Gemini upstream failure and safe retry (PR #13, PR #10)

Provider tests simulated HTTP 408, 429, 500, 502, 503, 504, interrupted connections and deadlines. A test demonstrates HTTP 504 followed by successful classification, while persistent 504 responses stop after a bounded retry cycle. Temporary provider interruptions use capped exponential backoff with jitter. PR #10 also ensures delayed retries do not occupy a worker slot during backoff. HTTP 401 credentials/configuration failures are treated as non-retryable in the immediate processing cycle and trigger safe deferral/cooldown instead of an uncontrolled request loop.

6.2 FR-02: Malformed AI output (PR #13)

Malformed JSON, unsupported classification decisions and missing flag reasons were injected. Passing tests such as `TestMalformedGeminiResponseExhaustionDefersWithoutPublishing` and `TestProviderFailuresAndMalformedDecisionsStaySubmitted` verify that unreliable model output is not automatically published. Deferred reviews remain eligible for controlled recovery.

6.3 FR-03: Worker crash, restart and stale lease (PR #13)

Persisted review state and lease ownership are tested under worker restart, repeated recovery scans and an expired earlier claimant. The final database-backed test `TestTriageExpiredWorkerCannotOverwriteNewerClaimResult` passed, as did `TestDeferredReviewRecoversAfterRestartAndClearsRetryMetadata` and the duplicate-recovery-scan test. Processing state is fenced to prevent an expired worker from overwriting a newer successful claim.

6.4 FR-04: Provider-wide cooldown (PR #13)

The persisted provider-cooldown integration test passed against MongoDB. It verifies that pending/new reviews are not processed during a provider-wide failure cooldown and become eligible again

after expiry. Recovery cycles are bounded even after repeated failure sequences.

6.5 FR-05: Atomic moderation resolution (PR #18)

Previously, the appeal-ticket update and associated review status change were separate writes, risking inconsistent states if processing failed in between. PR #18 moves appeal resolution and review updates into a MongoDB session transaction. On restore, a review receives publication metadata and its stale flag reason is cleared. Passing tests include `TestResolveAppealDoesNotResolveTicketWhenReviewIsMissing`, `TestAppealResolveIntegration/missing_review_rolls_back_ticket`, and the restore/delete/invalid-lock cases. A MongoDB replica set is required for these transactional semantics.

6.6 FR-06: Recoverable moderator action errors (PR #12)

Before PR #12, a failed ticket action could replace the entire detail page with an error message and hide the buttons needed to retry or release the ticket (`SCRUM-38`). The UI now keeps already-loaded ticket content visible and displays the action error above it. This was validated by the reported frontend build and source review; moderator component regressions were added to the broader test suite through PR #18. PR #14 additionally exposes active ticket-lock expiry in the interface (new feature, not a fix).

6.7 FR-07: Student-side degraded-service and storage failures (PR #11)

The teammate's test matrix documents seven targeted negative paths beyond the AI worker. When the course catalog is unavailable, the input remains operable as plain text and the backend still validates the course before accepting a review; a failed catalog request is not cached, allowing retry. A database error during catalog validation saves no feedback. Announcement fetch failure does not crash the dashboard. Unknown reference-code status returns a visible error while retaining the saved code. Corrupt notebook local-storage data starts empty rather than crashing, and a student-ID lookup failure tells the user to copy their reference code. These checks were exercised in the frontend/service and HTTP tests delivered in PR #11.

6.8 FR-08: Authentication throttling and concurrency failure paths (PR #15, PR #16, PR #17)

PR #15 prevents simultaneous invalid OTP verifications from all observing the same attempt count by reserving attempts atomically. PR #16 adds account-level login and OTP-request throttling, IP-level HTTP limits and stronger moderator/administrator password minima. The dedicated 100-request OTP experiment passed: five reservations and 95 attempt-limit rejections. PR #17 standardizes unknown-account and wrong-role failure paths, with dummy bcrypt work to reduce account enumeration differences. These are abuse/failure safety behaviors rather than claims of recovering a physically unavailable database.

ID	Injected condition	Result/safeguard	Evidence
FR-01	HTTP 504/network failure	Bounded retry, eventual success or deferral; worker released during wait	PR #10, PR #13
FR-02	Malformed JSON/decision	No invalid automatic publication	PR #13
FR-03	Restart/expired lease	Recovery and stale-worker fencing	PR #13
FR-04	Provider cooldown	Blocks pending work, resumes after expiry	PR #13
FR-05	Missing review during appeal resolve	Ticket not partially resolved; transaction rollback	PR #18

ID	Injected condition	Result/safeguard	Evidence
FR-06	Moderator action rejected	Loaded ticket and retry controls remain visible	PR #12
FR-07	Catalog, announcement or storage failures	No bad cache or invalid feedback; graceful UI state	PR #11
FR-08	OTP attempts/login abuse	Atomic reservation, throttled responses	PR #15–PR #17

Repository evidence: `04-failure-recovery/gemini-retry-recovery.txt`, `04-failure-recovery/provider-cooldown.txt`, `04-failure-recovery/stale-worker-protection.txt`, `04-failure-recovery/appeal-transaction-consistency.txt`, `03-concurrency/concurrent-otp-verification.txt`. The `04-failure-recovery/database-failure.txt` command found *no matching tests*; it is explicitly *excluded* as evidence of a live MongoDB disconnect/reconnect exercise.

7 Performance and Load Testing

7.1 Workload and setup

The application was started locally with a dedicated MongoDB `campusecho_performance_test` database. Autocannon generated **10 concurrent connections** against the authenticated endpoint `GET /api/v1/courses?search=cs`, for 30.16 seconds. In this environment the development-mock triage provider was configured, although the measured endpoint is read-only course search and does not invoke triage.

7.2 Measured baseline

Metric	Baseline
Requests completed / HTTP 200	18,135
Concurrent connections	10
Duration	30.16 s
Average request rate	604.5 requests/s
Mean response latency	16.07 ms
Median (p50) latency	10 ms
p90 latency	22 ms
p99 latency	133 ms
Maximum observed latency	990 ms
HTTP non-2xx responses	0
Transport errors / timeouts	0 / 0

Table 7: Autocannon results from the pushed machine-readable baseline.

All 18,135 completed responses were HTTP 200. The baseline represents local endpoint performance under a specific 10-connection workload; it does not establish deployment scalability or capacity across diverse client networks. The `throughput.average` JSON field represents bytes per second, whereas the 604.5 figure comes from `requests.average` and is the request rate.

7.3 Bottleneck hypothesis and missing comparison

The course-search query performs a case-insensitive regular-expression search across course ID, title and instructor and sorts results. This is a *candidate* for query-plan investigation, not a measured

bottleneck. No `explain()` analysis, optimization implementation, or comparable post-optimization load run was completed. Accordingly the handout's optimization/after-metrics requirement remains outstanding; no improvement percentage is reported.

Evidence: `05-performance/course-search-baseline.json`.

8 System and End-to-End Validation

The team reports manually completing the following workflows on the integrated application with all cases passing. This account is based on direct team confirmation, *not* on saved E2E screenshots or an uploaded E2E test log.

ID	Workflow exercised	Team-reported result
E2E-01	Student login, course browsing and review exploration	PASS
E2E-02	Student review submission and reference code generation	PASS
E2E-03	Review progression through automatic AI triage	PASS
E2E-04	Moderator claims and resolves a flagged item	PASS
E2E-05	Student submits an appeal, then moderator handles it	PASS

Table 8: Manual E2E checklist, as confirmed by the team.

9 Defects, Fixes and Regression Traceability

9.1 Detailed fixed-defect register

ID	Defect / discovery	Root cause and verified fix	Tracking / PR	Regression evidence
D01	Announcements absent immediately after registration (functional review).	Cookie-only login had no bearer token; frontend wrongly short-circuited. Allow cookie authentication without token.	`SCRUM-31`, PR #11	Cookie-only panel test.
D02	My Codes auto-refresh disabled after cookie session restored.	Refresh effect required token. Remove token-only condition.	`SCRUM-31`, PR #11	Cookie-only refresh test.
D03	Course suggestions missing after cookie login.	Catalog hook exited without token. Use cookie-aware requests.	`SCRUM-31`, PR #11	Dropdown regression.
D04	New reference code not auto-saved; possible cross-student ID reuse.	Student lookup was token-only; unsafe cache across cookie-only sessions. Fetch with cookie and avoid caching unscoped identity.	`SCRUM-31`, PR #11	Save-code and student-switch tests.
D05	Review-flooding through case/spacing variations or unlisted course IDs (**adversarial**, team bug 4).	Free-text course IDs split course-specific abuse counts. Server canonicalizes through catalog; rejects unlisted entries and uses catalog title.	`SCRUM-39`, PR #11	Go course check; submit form tests.
D06	Explore with no results shows a malformed-response error (team bug 7).	Nil Go slice serialized to JSON `null`; return an explicit empty array.	`SCRUM-36`, PR #11	Empty-search test; screenshots.
D07	Ticket disappears when moderator action fails (team bug 9).	Action error replaced previously loaded page state. Keep ticket/buttons, show inline error.	`SCRUM-38`, PR #12	Build and source diff; moderator UI tests in PR #18.
D08	Concurrent OTP requests bypass attempt limit (**concurrency**).	Read-before-write attempt count raced. Atomic conditional MongoDB reservation before comparison.	`SCRUM-28`, PR #15	OTP atomic-limit test; concurrency log.

ID	Defect / discovery	Root cause and verified fix	Tracking / PR	Regression evidence
D09	Repeated login/OTP requests insufficiently throttled; privileged passwords too short (**security**).	Missing sufficient per-account/IP protection and password minima. Added limits, lockout and 12-char privileged minimum.	`SCRUM-29`, PR #16	Login/OTP rate tests.
D10	Authentication responses expose account/role existence (**security**).	Distinguishable error paths and missing comparable password work. Generic failure and dummy bcrypt work.	`SCRUM-30`, PR #17	Unknown-email/wrong-role auth regressions and handler tests.
D11	Restored reviews retain old flag reason / no publish date (team bug 11).	Restore updated status without clearing metadata. Update `publishedAt` and clear `flagReason`.	Team bug 11, PR #18	Restore integration test.
D12	Appeal and review updates can diverge after partial failure (**failure**).	Two independent MongoDB writes. Perform both inside one transaction and roll back on missing review.	PR #18; Jira to link	Missing-review rollback test.
D13	Gemini timeout/malformed response can strand reviews or mispublish invalid output (**failure**).	Recovery metadata, retry classification and lease safety were insufficient. Persistent cooldown, bounded recovery and fenced writes.	PR #13; Jira to link	Gemini retry/recovery tests.
D14	Duplicate misclassification for short/Unicode reviews; retry backoff occupies workers (triage review).	Duplicate heuristics and sleep-based retry scheduling. Preserve Unicode, require six words, bound comparison to 200 same-course reviews and schedule retry asynchronously.	PR #10; `SCRUM-7` workstream	Duplicate/brigading unit tests and triage retry tests.

Counting discipline. D01–D04 are four user-visible symptoms of the shared cookie-session authorization assumption in PR #11; they should not be presented as four unrelated root causes. D05, D06, D07, D08, D09, D10, D11, D12, D13 and D14 represent other distinct fix areas. D05 (adversarial), D08 (concurrency), D09–D10 (security) and D12–D13 (failure) fulfill the special discovery-method requirement, with directly linked PRs and tests.

9.2 Pull-request map: #10–#18

PR	Primary delivered change	Classification
PR #10	Duplicate/brigading correctness; nonblocking retry scheduling and tests	Bug/reliability
PR #11	Cookie sessions, course ID abuse, empty Explore, 54 tests	Bugs + tests
PR #12	Ticket remains visible after a failed moderator action	Bug fix
PR #13	Gemini provider retries, cooldown, persisted recovery and leases	Bug/reliability
PR #14	Live moderator ticket lock-expiry countdown	**Feature**
PR #15	Atomic OTP reservation under concurrent verification	Security bug
PR #16	Login/OTP throttling, stronger privileged passwords	Security bugs
PR #17	Uniform account/role login failures; dummy password work	Security bug
PR #18	Atomic appeal resolution, restoration metadata and moderator tests	Consistency bugs

Table 10: All nine merged PRs in the requested range; feature #14 is not counted as a defect.

9.3 Regression and screenshot methodology

The PR #11 teammate report records nine tests that were checked to fail on the original code and pass after fixes (seven cookie-session cases, two course-validation cases), with before/after website screenshots of announcements, dropdown selection, reference saving, course rejection and empty Explore results. The moderator teammate report records a build-verified ticket-error fix and deliberately failing *break-it* authorization/validation mutations that were reverted. PR #15, PR #16, PR #17 and PR #18 supply executable security/concurrency/transaction regression checks. These

make the fixes auditable rather than relying on a verbal defect list alone. A consolidated link from each PR to a Jira ticket should be added where the repository lacks one.

10 Cloud Deployment and Operational Validation

10.1 Deployment architecture

CampusEcho was deployed on Amazon Web Services (AWS) using a single EC2 instance, chosen to keep the deployment simple and inexpensive. The frontend and backend share the instance. Nginx serves the compiled frontend and forwards `/api/` requests to the Go backend listening locally on port 8080. MongoDB Atlas provides managed database hosting.

Component	Deployment configuration
Cloud provider	AWS
Region	Asia Pacific (Sydney), `ap-southeast-2`
Operating system	Ubuntu Server 26.04 LTS, `x86_64`
Compute	EC2 `t3.micro`, 2 vCPU, 1 GiB RAM
Storage	8 GiB General Purpose SSD (`gp3`)
Frontend	React + TypeScript + Vite production build
Web server / reverse proxy	Nginx
Backend	Go REST API
Process management	systemd (`campusecho.service`)
Database	MongoDB Atlas M0 Free cluster
AI integration	Gemini API for triage and brigading detection
Email service	Gmail SMTP for OTP delivery

Table 11: Deployed architecture and configuration.

10.2 Deployment configuration

The following steps were completed:

1. Created an Ubuntu EC2 instance and configured its security group.
2. Connected over SSH and cloned the CampusEcho repository.
3. Installed Go, Node.js, npm, Git, Nginx and required dependencies.
4. Downloaded the backend Go modules and built the API executable.
5. Configured MongoDB Atlas connectivity through environment variables.
6. Supplied Gmail SMTP and Gemini API credentials through a backend `.env` file, restricted to the application owner by file permissions.
7. Created a systemd service that starts the backend automatically and restarts it after failures.
8. Built the frontend with `npm ci` and `npm run build`.
9. Configured Nginx to serve the production build and proxy API requests to the backend.
10. Verified the Nginx configuration and reloaded the service.

The backend is managed by `campusecho.service`, and Nginx is enabled as a system service.

10.3 Network and security configuration

Port	Purpose	Access
22	SSH administration	Restricted to the administrator's IP address
80	HTTP web access	Public
443	HTTPS	Inbound rule configured

Table 12: EC2 security-group inbound rules.

MongoDB Atlas network access is restricted to the EC2 public IP address rather than open to all addresses. Application secrets are held in the backend environment file and are not included in the frontend build.

Deployment URLs. The application is reachable over HTTPS at `https://3.27.238.157.nip.io/`. The validation results in this section were recorded against the original HTTP address `http://3.27.238.157`.

10.4 Operational validation

Validation	Team-reported result
Go backend compilation	Successful
Frontend production build	Successful
MongoDB Atlas connectivity	Successful
Backend systemd service	Active and running
Nginx configuration test	Successful
Nginx frontend serving	HTTP 200 OK
Backend health endpoint through Nginx	HTTP 200 OK
Student signup and OTP verification	Tested successfully
Login workflow	Tested successfully

Table 13: Deployment smoke tests.

The health endpoint returned `{"status":"ok"}` through Nginx, and the frontend returned the production HTML page. These checks establish that the stack is reachable and that registration, OTP delivery and login work on the deployed instance.

Deployed commit: `be9b8085aa538a7d5cfac4e64760475ce5eb2b95`.

10.5 Deployment access and maintenance

The deployed application is available at `https://3.27.238.157.nip.io/`. Useful maintenance commands:

```
sudo systemctl status campusecho
sudo journalctl -u campusecho -n 50 --no-pager
sudo systemctl status nginx
sudo nginx -t
```

To deploy subsequent changes, pull the required repository updates, rebuild the Go backend and the frontend, restart the backend service, and reload Nginx if its configuration changed.

11 Cost Estimation

11.1 Assumptions

The deployment was designed to minimize infrastructure cost by using a small EC2 instance, the MongoDB Atlas free tier and API free-tier allowances where available. The estimate assumes:

- one EC2 `t3.micro` instance running continuously for approximately 730 hours per month;
- one 8 GiB `gp3` storage volume and one public IPv4 address;
- a MongoDB Atlas M0 Free cluster;
- low-volume, student-project traffic;
- no load balancer, NAT Gateway, RDS instance or other paid infrastructure.

11.2 Estimated monthly infrastructure cost

Resource	Usage assumption	Monthly cost (USD)
AWS EC2 `t3.micro`	730 h at approx. $0.0132/h	$9.64
EBS `gp3` storage	8 GiB at an illustrative $0.08/GB-month	$0.64
Public IPv4 address	730 h at $0.005/h	$3.65
MongoDB Atlas M0	Free cluster	$0.00
Gemini API	Free tier, subject to eligibility and limits	$0.00 initially
Gmail SMTP	Existing account with app password	No additional charge assumed
Estimated baseline total	Before taxes, data transfer and extra usage	**$13.93**

Table 14: Estimated monthly infrastructure cost.

The EC2 and storage figures are planning estimates, not an AWS invoice. The storage rate is illustrative and must be checked against the Sydney regional rate. Actual charges depend on AWS pricing, free-tier benefits, credits, usage and taxes.

11.3 AI API cost considerations

CampusEcho uses a Gemini text model for AI triage and `gemini-embedding-001` for semantic embeddings and brigading detection. Both have free-tier availability subject to Google's eligibility and usage limits. If paid usage applies, the text-model rate depends on the model configured and is listed on Google's Gemini API pricing page, and Gemini Embedding 001 is listed at approximately $0.15 per million input tokens. Actual expenditure depends on the number of reviews processed, token counts and embedding requests. No measured monthly AI consumption figure is claimed in this report.

11.4 Cost optimization measures

1. A single `t3.micro` instance instead of multiple servers.
2. MongoDB Atlas M0 Free instead of a paid database cluster.
3. Frontend and backend served from the same instance.
4. Nginx as the reverse proxy instead of a separate load balancer.
5. No NAT Gateway or other unnecessary paid networking resources.
6. Gemini free-tier availability used where eligible.

7. Storage kept at 8 GiB.

11.5 Actual cost and usage verification

The deployment is configured, but the estimate has *not* been compared with billing records. The AWS account held approximately $100 in promotional credits at deployment, with about 182 days of validity remaining. At the estimated $13.93/month, six months of operation would cost roughly $83, which the credits would cover. Credits offset eligible charges only and do not make the services permanently free. Actual charges should be verified in AWS Billing and Cost Management, Gemini usage in Google AI Studio, and database usage in MongoDB Atlas. Higher traffic, increased AI usage, extra storage, data transfer or paid upgrades would raise the monthly cost.

Conclusion. The architecture gives a low-cost deployment suitable for a student demonstration, with a baseline of about $13.93 per month before credits, taxes and data transfer.

12 Planning, Requirements Traceability, Jira and AI Engineering Log

12.1 Jira project audit and issue-status snapshot

The team's Jira Software project is **CS455** (project key `SCRUM`). We inspected the complete project issue list and the associated *SCRUM board* on 6 November 2026. At that snapshot, the project had **36 issues**, all recorded in Jira as the *Task* issue type: **34 Done**, **two To Do**, and none In Progress. The issue-key gaps `SCRUM-17`–`SCRUM-18` and `SCRUM-33`–`SCRUM-35` do not represent missing results from the query; the project search returned all 36 existing issues. Some Task records describe defects; they are classified as defect work in this report based on their descriptions rather than their Jira issue type.

Jira group	Issues	Done / To Do
Functional requirements (`SCRUM-1`–`13`)	13	13 / 0
Non-functional requirements (`SCRUM-14`–`16`, `19`–`21`, `40`–`41`)	8	6 / 2
Additional application features (`SCRUM-22`–`27`)	6	6 / 0
Testing, defects and enhancements (`SCRUM-28`–`32`, `36`–`39`)	9	9 / 0
Total	**36**	**34 / 2**

Table 15: Jira issue inventory at the 6 November 2026 snapshot.

Issues are assigned across all six team members: M. Amara Gayathri (9), Vaishnavi B (7), Reddi Pallavi (6), Kishore Senthilkumar (5), Obul Reddy (5), and Poorvie Sadagopan (4). These are assignment counts, not claims about individual code contribution or effort.

Open Jira requirements. Only two issues remain *To Do*: `SCRUM-40` (NFR-4: demonstrate review-submission response within one second) and `SCRUM-41` (NFR-5: demonstrate triage within 60 seconds). The original `SCRUM-17` and `SCRUM-18` records were deleted and recreated as `SCRUM-40` and `SCRUM-41`. `SCRUM-14` (NFR-1) and `SCRUM-20` (NFR-7) are now marked Done in Jira; their completion status is reported separately from any independently measured acceptance evidence.

12.2 Sprint and planning evidence

The project has one visible board (*SCRUM board*, board ID 1) and one visible sprint, *Sprint 1 – Core CampusEcho Wor*, whose goal is to establish authentication, anonymous review submission, status tracking, appeals, moderation, and traceability. Jira currently marks this sprint **Future**, not Active or Closed. Ten issues have this sprint field, whereas 26 issues have no sprint assignment. Thus the

34 Done issues *do not* establish a completed sprint, sprint velocity, a burndown curve, or a validated sprint start/end date. We present issue/workflow progress separately from sprint execution evidence.

The following is an *observed record chronology*, not a retroactively invented Gantt schedule:

Date	Jira/project record
26 Sep 2026	Initial FR-1–FR-13 and NFR-1–NFR-8 tasks created (SCRUM-1–SCRUM-21).
22 Oct 2026	My Codes, course catalog, and announcements added as feature tasks (SCRUM-22–SCRUM-24).
28 Oct 2026	Deleted-review retention, admin metadata, and student appeals integration tracked (SCRUM-25–SCRUM-27).
5 Nov 2026	Authentication/security fixes, cookie-session fixes, frontend testing and Explore regression entered (SCRUM-28–SCRUM-32, SCRUM-36).
6 Nov 2026	Countdown feature, moderator error-state fix, and course-name flooding fix recorded (SCRUM-37–SCRUM-39); NFR-4 and NFR-5 re-entered as SCRUM-40 and SCRUM-41; latest test evidence collected.

Table 16: Milestones substantiated by Jira creation dates and final repository evidence.

Planning limitation: a planned-versus-actual Gantt chart and complete sprint performance metrics require separately verified dates or team planning artefacts. They are not inferred from Jira statuses.

12.3 Functional requirements: Jira–implementation–test traceability

All thirteen primary functional Jira tasks are *Done*. The following matrix connects their acceptance areas with relevant merged GitHub PRs and representative code/tests. A passing linked test verifies its assertions; it does not automatically establish external AI quality or cloud deployment.

Req.	Jira issue	Implementing PR(s)	Representative implementation / verification
FR-1	SCRUM-1	PR #1, PR #2	Student registration, bcrypt/session handling, administrator provisioning; `internal/auth` and HTTP auth tests.
FR-2	SCRUM-2	PR #1, PR #2, PR #5	Student/moderator role-gated routes; `TestSecurityModeratorRouteTokensAndBodies`.
FR-3	SCRUM-3	PR #2, PR #11	Anonymous review persistence/reference code; course-catalog validation and `course_check_test.go`.
FR-4	SCRUM-4	PR #2, PR #11	Published-review exploration/ratings; `explore_empty_test.go` and feedback-handler tests.
FR-5	SCRUM-5	PR #2, PR #9, PR #11	Reference-code lookup; My Codes notebook and status-refresh component tests.
FR-6	SCRUM-6	PR #7, PR #13	Asynchronous Gemini triage; provider, worker, decision-validation and recovery tests.
FR-7	SCRUM-7	PR #8, PR #10, PR #11	Same-course Unicode-aware duplicate checks; `triage_duplicate_test.go`; canonical course IDs.
FR-8	SCRUM-8	PR #8, PR #10	Threshold-based semantic brigading; `brigading_test.go`. Requires `BRIGADING_MODE=gemini`; development no-op is not detection.
FR-9	SCRUM-9	PR #2, PR #5	Single appeal per flagged review, ticket creation and student-facing appeal integration tests.
FR-10	SCRUM-10	PR #3, PR #5	Moderator queue ordering/filtering; queue-handler and MongoDB queue integration tests.
FR-11	SCRUM-11	PR #3, PR #5	Atomic MongoDB claim; `TestQueueClaimConcurrentRequests`: 50 claimants, 20 rounds.
FR-12	SCRUM-12	PR #4, PR #5, PR #14	Ticket release/expiry handling and visibility of lock deadline; appeal tests. Countdown (PR #14) is a feature addition.
FR-13	SCRUM-13	PR #4, PR #5, PR #18	Lock-holder-only resolution, restore/delete and transactional consistency; `TestAppealResolveIntegration`.

12.4 Non-functional requirements and verification gaps

This table deliberately distinguishes *Jira workflow status* from *independently demonstrated acceptance*. In particular, SCRUM-15 and SCRUM-21 are marked Done in Jira, but their security properties still need configuration-specific verification.

Req.	Jira	Jira state	Related PR(s)	Verification and outstanding detail
NFR-1	SCRUM-14	Done	PR #2	Review records are designed without direct student identity fields. Jira is Done; a separate schema-inspection artefact is not established by this section.
NFR-2	SCRUM-15	Done	PR #19	Dedicated triage MongoDB connection and restricted interface added. PR notes that the least-privilege database-permission integration check was skipped without restricted credentials.
NFR-3	SCRUM-16	Done	PR #3, PR #5	MongoDB atomic-claim experiment passed: one winner in each of 20 rounds of 50 simultaneous claims; 03-concurrency/concurrent-ticket-claim.txt.
NFR-4	SCRUM-40	To Do	PR #7	Submission dispatch is asynchronous, but no measured proof of the required sub-one-second *submission* response was supplied. Course-search load is not a substitute.
NFR-5	SCRUM-41	To Do	PR #7, PR #13	Triage worker/recovery paths are tested; the normal-load *within-60-seconds* criterion remains unmeasured.
NFR-6	SCRUM-19	Done	PR #7, PR #10, PR #13	Retries, persistent cooldown, worker leases and safe deferral passed; 04-failure-recovery/gemini-retry-recovery.txt.
NFR-7	SCRUM-20	Done	–	Jira is Done, but this section does not contain a verified measured false-positive rate for the specified 50-review labelled set.
NFR-8	SCRUM-21	Done	PR #1, PR #16, PR #17	Bcrypt and backend role checks tested; the live HTTPS-only deployment condition needs teammate deployment evidence.

12.5 Additional features, defect tickets and QA work

The additional fifteen Jira tasks are all Done. The table records *explicitly attributable* PRs where available. A dash means the Jira issue did not establish one unique PR; it is not a claim that no code exists.

Jira	Tracked feature / problem	PR	Evidence / relationship
SCRUM-22	Student reference notebook (My Codes)	PR #6, PR #9	Notebook service/component tests.
SCRUM-23	Administrator course catalog and student selection	PR #6, PR #9	Catalog API and dropdown tests.
SCRUM-24	Course-scoped and general announcements	PR #6, PR #9	Announcements service/panel tests.
SCRUM-25	Purge moderator-deleted reviews after 72 hours	–	Expiry/purge worker and database tests; direct PR mapping not recorded on issue.
SCRUM-26	Review metadata integration and admin-page structure	–	Admin-page acceptance criteria; direct PR mapping not recorded on issue.
SCRUM-27	Student appeals integration and status visibility	–	Student appeal workflow; direct PR mapping not recorded on issue.
SCRUM-28	Concurrent OTP attempt-limit bypass	PR #15	100-attempt test; five reserved and 95 blocked.

Jira	Tracked feature / problem	PR	Evidence / relationship
SCRUM-29	Login/OTP rate limits and stronger passwords	PR #16	Auth and HTTP rate-limit tests.
SCRUM-30	Account/role enumeration through login responses	PR #17	Uniform failure and dummy bcrypt-work tests.
SCRUM-31	Cookie-only sessions break announcements, course suggestions, My Codes	PR #11	Cookie-restored-session regressions.
SCRUM-32	Frontend test framework and 50 feature tests	PR #11	Vitest/React Testing Library, eight frontend test files.
SCRUM-36	Empty Explore results encoded as `null`	PR #11	`explore_empty_test.go`.
SCRUM-37	Active-appeal lock-expiry countdown (feature)	PR #14	Deadline handling and build tests.
SCRUM-38	Failed moderator action hides ticket details	PR #12	Ticket page fix and build validation.
SCRUM-39	Course-ID spelling variants bypass flooding guard	PR #11	`course_check_test.go`; UI validation.

12.6 Defect-to-Jira-to-PR-to-regression audit trail

The following representative chains support the Deliverable 3 defect criterion and include fixes discovered via adversarial, concurrency and failure-path testing. Section 9 gives the expanded technical root-cause analysis. All linked PRs below were merged.

Jira	Failure mode	Fix PR	Regression / inspection
SCRUM-28	Concurrent OTP guesses exceed limit	PR #15	`TestOTPVerificationReservesAtMostFive ConcurrentAttempts`.
SCRUM-29	Login/OTP abuse; weak privileged passwords	PR #16	`TestLoginRateLimits RepeatedFailuresAnd ResetsAfterSuccess`; password and OTP request tests.
SCRUM-30	Account discovery via distinct login failures	PR #17	Auth tests for unknown email, wrong role and generic failures.
SCRUM-31	Cookie-restored sessions lose student features	PR #11	Seven failing-before/passing-after session regressions (teammate report).
SCRUM-36	Empty Explore crashes client contract	PR #11	`explore_empty_test.go`.
SCRUM-38	Ticket vanishes after moderator action error	PR #12	UI failure-state fix; clean build.
SCRUM-39	Course variants circumvent spam accounting	PR #11	Course catalog lookup, canonical IDs and adversarial review tests.
SCRUM-19	AI provider failure risks invalid publication	PR #13	Gemini retry/recovery, malformed-output and stale-lease regressions.
SCRUM-13	Appeal and review writes may disagree	PR #18	Transactional `TestAppealResolveIntegration`, including missing-review rollback.

12.7 AI Engineering Log and Human Validation

AI tools were used during the development of CampusEcho for code analysis, debugging, implementation, regression testing, and documentation. The following log records representative AI-assisted engineering activities, the prompts provided, the AI-generated solutions, and the human verification performed.

AI-generated changes were reviewed and tested before being incorporated into the project.

Task / AI Tool	Prompt or Problem	AI Contribution	Verification and Evidence
Gemini AI Triage Recovery GitHub Copilot	Implement automatic recovery for reviews whose Gemini classification fails because of HTTP errors, network interruptions, malformed responses, or invalid API credentials. Preserve existing review statuses, worker pools, and recovery architecture while ensuring bounded retries, provider cooldowns, and concurrency safety.	Implemented persistent retry scheduling with sanitized failure categories, retry timestamps, and per-cycle attempt counters. Extended the recovery scheduler to redispatch eligible reviews, added provider-level cooldowns, and introduced tests for recovery, worker restarts, active leases, queue saturation, and stale-worker protection.	Go triage and configuration tests passed. Race-detector tests passed. MongoDB integration tests were initially skipped because the test database was unavailable; subsequent project-level integration tests passed. PR #13; `SCRUM-6`, `SCRUM-19`.
Duplicate Review Detection Claude	Investigate whether duplicate normalization removes non-English characters and incorrectly flags short, generic comments such as "Good course". Verify that the bugs exist before implementing changes.	Confirmed that the original normalization retained only ASCII characters. Modified normalization to preserve Unicode letters, numbers, and combining marks. Added a minimum length of six words for both reviews before duplicate comparison and created regression tests for Hindi reviews and short comments.	Focused regression tests were added. The PR reports successful triage race-detector tests and static-analysis checks. PR #10; `SCRUM-7`.
Cookie-Session Bug Diagnosis Claude	Investigate why announcements, course suggestions, and My Codes fail immediately after registration or when a session is restored using only an HttpOnly cookie.	Identified frontend checks that incorrectly required an in-memory access token. Modified requests to use a bearer token when available and cookie-based authentication otherwise. Identified four related failures sharing the same underlying cause.	Regression tests reproduced the failures before the changes and passed afterward. Manual before/after testing was performed. PR #11; `SCRUM-31`.
Frontend Testing Framework Claude	Establish automated frontend testing for course selection, announcements, and the student reference notebook.	Configured Vitest and React Testing Library. Generated unit, component, and integration tests covering normal workflows, invalid responses, browser storage failures, and regression scenarios.	50 frontend tests and four backend tests were reported for these features. Tests were executed locally. PR #11; `SCRUM-32`.
Course-ID Review Flooding Claude	Prevent users from bypassing course-specific review-flooding checks through inconsistent course-ID capitalization or spacing.	Implemented server-side course catalog validation. Canonical course identifiers and titles are retrieved from the catalog instead of trusting arbitrary client-submitted values. Unlisted courses are rejected.	Backend and frontend tests verified course validation. Manual before/after screenshots confirmed the corrected behavior. PR #11; `SCRUM-39`.
Empty Explore Results Claude	Investigate why searching for reviews with no matching results produces an unexpected-response error.	Identified that a nil Go slice was serialized as JSON `null`. Modified response handling to return an empty JSON array, `[]`.	Backend regression tests passed. Manual frontend verification confirmed the empty-results view. PR #11; `SCRUM-36`.

Task / AI Tool	Prompt or Problem	AI Contribution	Verification and Evidence
End-to-End Testing Assistance Claude	Provide a reproducible procedure for testing the student workflow against the local frontend, backend, and database.	Provided testing instructions covering registration, course selection, review submission, reference-code saving, and review exploration.	The workflow was executed locally. Before/after screenshots and test observations were recorded for affected features. PR #11.
Jira and Merge Assistance Claude	Assist with documenting bug fixes in Jira and resolving merge conflicts encountered during pull-request preparation.	Helped prepare issue descriptions, identify a README merge conflict, and determine the intended resolution.	Jira descriptions were reviewed. The README conflict was corrected, and the pull request was merged. PR #11.

13 Conclusion and Limitations

Evidence pushed to the project repository supports successful automated backend/frontend test execution, statement coverage above 80%, a passing full Go race run, server-side security regressions, atomic ticket-claim and OTP reservation invariants, triage worker recovery safeguards, transaction consistency, and successful authenticated course-search performance measurement. The team additionally reports five successful manual E2E workflows.

Limitations and Future Work Although the major functional requirements and testing activities have been completed, two non-functional performance requirements remain pending formal verification:

- **NFR-4 – Sub-second review submission (`SCRUM-40`):** The system uses asynchronous AI triage to avoid blocking review submission. However, the requirement that submission consistently completes within one second has not yet been verified through a dedicated response-time benchmark.
- **NFR-5 – Triage within 60 seconds (`SCRUM-41`):** The asynchronous triage pipeline is implemented and tested, but its ability to process reviews within 60 seconds under representative normal load has not yet been formally measured and verified.

Both issues remain in the `To Do` state in Jira and are identified as future performance-validation tasks.

The merged PRs and team reports establish at least five independently meaningful fixed defects, including security, concurrency and failure-triggered examples, with tests and Jira links where available.The report verified cloud deployment, costing, final dashboard, a consolidated six-member AI log and any missing Jira IDs.

A Evidence Folder Index

- Automated test logs and coverage
- Security and adversarial test logs
- Concurrency experiments
- Failure/recovery and database-consistency tests
- Performance baseline JSON